

HoneyPot System

Complete Technical Handbook

A code-grounded guide to the honeypot, SOC dashboard, detection pipeline, and operations model

Repository baseline: 6cad5b7 (stabilize browser regression suite)
Document date: 29 September 2026
Deployment status: **Not deployed to the public Internet**
Validation status: **Mostly validated on native Windows runtime**

This handbook distinguishes implemented behavior from untested integrations and future work. It does not contain secrets.

Contents

1	Project Overview	1
1.1	Users and use cases	2
2	Cybersecurity and Honeypot Basics	3
3	High-Level Architecture	4
3.1	Every arrow	4
4	Complete Data Flow	5
5	Frontend: Next.js SOC Dashboard	6
5.1	SSE client	6
6	Backend: FastAPI	7
7	Authentication	8
8	Role-Based Access Control	9
9	Password Reset	10
9.1	Webhook signing	10
10	Rate Limiting	11
11	Redis and Redis Streams	12
12	Worker	13
13	PostgreSQL, Migrations, and Data Model	14
14	SSH Honeypot	15
15	HTTP Honeypot and Ingestion Pipeline	16
16	Detection Engine	17
17	Scoring and Incident Correlation	18
18	Alerts and Notifications	19
19	Server-Sent Events	20
20	Machine-Learning Pipeline	21
21	Threat Intelligence	22

22 Security Architecture	23
22.1 Containers and secrets	24
23 Error Handling and Resilience	25
24 Technology Stack and Rationale	26
25 Project Directory Structure	27
26 API Documentation	28
27 Testing and Evidence	29
28 Deployment	30
29 Limitations	31
30 Future Improvements	32
31 Interview Preparation	33
32 Glossary	35
Source Basis and Reading Order	36

List of Figures

3.1	Implemented logical architecture. Database and Redis are internal; the dashboard is the analyst UI.	4
4.1	Event lifecycle	5
7.1	Login and token issuance	8
9.1	Password-reset flow	10
13.1	Simplified implemented entity relationships	14
19.1	SSE update path	20
20.1	Implemented ML lifecycle	21
22.1	Security boundaries: only intended interfaces cross layers	23
28.1	Target deployment architecture, not a statement of current deployment	30

List of Tables

2.1	Core terms	3
13.1	Important persisted entities	14
16.1	All 13 implemented detection rules	17
22.1	Threat-to-defense mapping	23
24.1	Actual technology stack	26
25.1	Important repository areas	27
26.1	Important implemented API surface	28
27.1	Recorded validation evidence	29
31.1	Interview questions and accurate answers	33
32.1	Glossary	35

Chapter 1

Project Overview

What is it? HoneyPot System is a production-style cybersecurity monitoring platform. Two decoy services collect attacker interaction, FastAPI validates and persists it, deterministic rules create explainable detections and incidents, a Redis-backed worker enriches work, and a Next.js dashboard presents the SOC view.

Why do we need it? A small organization needs visibility into hostile behavior without placing a real production service at risk. A decoy offers controlled evidence of probing, credential guessing, malicious commands, and web attacks.

Project in one sentence

An isolated SSH and HTTP deception system turns authenticated attacker telemetry into scored, correlated SOC incidents visible in real time.

Project in 30 seconds

Attackers interact with fake SSH or HTTP surfaces. The decoys never execute their commands; instead they send normalized events to an authenticated API. Rules detect behaviors such as traversal, SQL injection, brute force, and reverse-shell attempts. PostgreSQL stores evidence, Redis sends background jobs to a worker, and the Next.js SOC dashboard receives live updates through Server-Sent Events (SSE).

Project in 2 minutes

A Security Operations Center (SOC) is the operational function that watches security telemetry, triages suspicious activity, and tracks incidents. This project implements a compact SOC workflow around safe honeypots. SSH and HTTP decoys capture interaction and post it using a shared sensor key. FastAPI validates the event, creates the attacker/session/event records, evaluates rules synchronously, calculates a transparent score, and correlates related detections from the same IP into a 15-minute unresolved incident. It then publishes a Redis Stream job. The worker optionally enriches the IP, classifies the incident with a persisted logistic-regression baseline, and delivers in-app or configured webhook notifications. Analysts authenticate through a Next.js backend-for-frontend (BFF), investigate events and incidents, acknowledge or resolve them, and see new telemetry by SSE. It is locally validated, but it is **not publicly deployed**.

1.1 Users and use cases

Intended users are SOC analysts and administrators. Analysts investigate events, alerts, attackers, sessions, maps, and incidents. Administrators manage users, rules, and audit logs. The platform is an educational/portfolio system and a controlled sensor, not a replacement for a mature SIEM or an unrestricted malware lab.

Chapter 2

Cybersecurity and Honeypot Basics

What is it? Cybersecurity monitoring collects telemetry—records about connections, requests, logins, and commands—then turns it into decisions. A SOC is the people, process, and tooling that performs this monitoring. A SIEM-like system centralizes evidence and helps search, correlate, and alert.

Table 2.1: Core terms

Term	Meaning in this project
Event	One observed action, such as an HTTP request or SSH command.
Detection	A rule match attached to an event, with evidence and a score.
Alert	An attention signal for a qualifying incident.
Incident	Correlated, analyst-managed security case spanning one or more events.
Threat	A potential harmful actor or capability; it is broader than a single event.
Indicator/IOC	Observable clue such as an IP, path, scanner user-agent, or command token.

How it works here. A honeypot is a deliberately exposed decoy. It is valuable because interaction should be unusual and therefore useful telemetry. This project captures interaction rather than retaliating, executing code, or claiming attacker attribution. Threat intelligence is optional IP enrichment, not a prerequisite for detection.

Chapter 3

High-Level Architecture

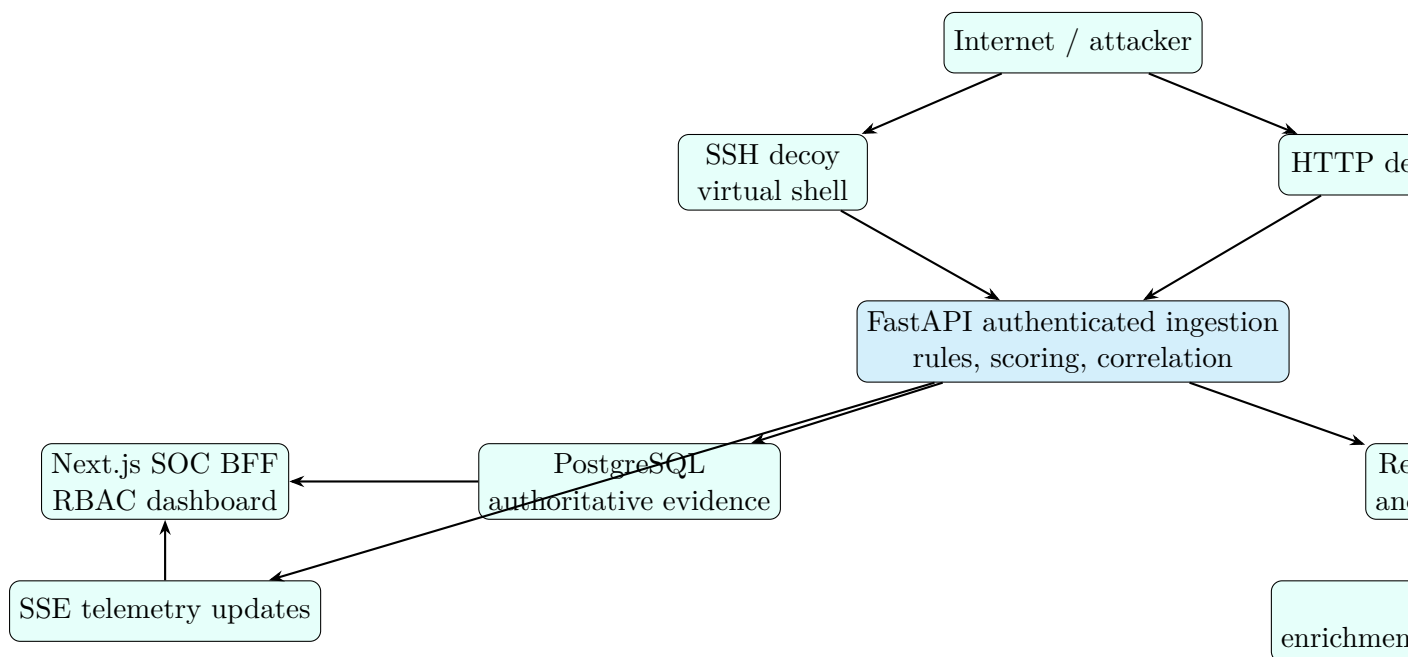


Figure 3.1: Implemented logical architecture. Database and Redis are internal; the dashboard is the analyst UI.

How it works here. The API is the trust boundary between sensors and data services. Decoys supply a sensor key but no database or Redis credentials. PostgreSQL stores transactional evidence before background work is queued. Redis decouples slow enrichment from the ingestion response. The Next.js BFF holds browser tokens in HTTP-only cookies and proxies management requests.

3.1 Every arrow

Attacker-to-decoy is untrusted interaction. Decoy-to-API is a normalized, sensor-key-authenticated event. API-to-PostgreSQL is durable event/detection/incident persistence. API-to-Redis is an XADD job plus a publish notification. Redis-to-worker is consumer-group processing. API/database-to-dashboard is authenticated management data through the BFF; API-to-browser is a one-way SSE event stream.

Chapter 4

Complete Data Flow

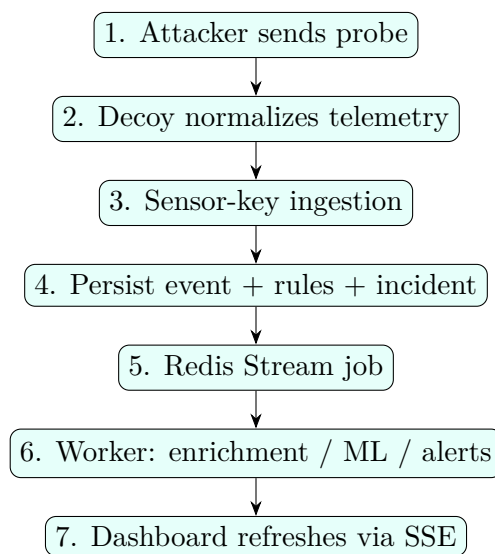


Figure 4.1: Event lifecycle

How it works here. The API first rejects oversized bodies and invalid sensor credentials. `ingest_event` is idempotent on the supplied event UUID. It upserts the node, attacker, and session; builds 60-second context; evaluates rules; records detections; creates specialized HTTP/command/credential records; correlates an unresolved 15-minute IP incident; and commits. Only after commit does it enqueue background work. If Redis is unavailable, PostgreSQL remains authoritative and the API returns the stored result; worker health exposes the outage.

Chapter 5

Frontend: Next.js SOC Dashboard

What is it? The only production frontend is `apps/web`, a Next.js 16 / React 19 / TypeScript application. It has protected dashboard, events, incident, attacker, session, honeypot, analytics, alerts, threat-intelligence, settings, users, rule, and audit-log routes.

Why do we need it? Next.js provides route structure and server/client rendering boundaries; TypeScript constrains API-shape mistakes; React Query caches and invalidates data. Tailwind supplies consistent utility styling; Recharts renders overview charts.

How it works here. The browser calls Next BFF routes under `/api/management/*`, not FastAPI directly. The BFF forwards the authenticated request. Client views use `api<T>()`, which applies a 10-second timeout and turns network failure into an `ApiError(503)`. The overview explicitly renders loading, empty, error, and retry states. There is a global Next error boundary. No production mock-data fallback exists.

5.1 SSE client

The `LiveListener` opens `EventSource('/api/management/stream/events')`. On a `telemetry` event it invalidates overview, events, incidents, and alerts queries. On error it closes the stream, displays reconnecting state, waits one second, and reconnects; the UI also exposes a `Reconnect` button.

Listing 5.1: Actual SSE client behavior, condensed from `live-listener.tsx`

```
stream.addEventListener("telemetry", () => client.invalidateQueries({queryKey:["overview"]}));
stream.onerror = () => { stream.close(); setStatus("reconnecting");
  setTimeout(() => setAttempt(value => value + 1), 1000); };
```

Chapter 6

Backend: FastAPI

What is it? `apps/api` is a FastAPI application with Pydantic request models, SQLAlchemy ORM, Alembic migrations, and routers for health, authentication, ingestion, and management.

How it works here. A request flows through CORS and security middleware, a route/dependency, Pydantic validation, business logic, database/Redis access, then a JSON or streaming response. Middleware adds `nosniff`, `DENY` framing, and `no-referrer` headers. Ingestion has content-size and per-minute rate limiting. Management routes require an authenticated user. Health routes expose API, database, Redis, worker heartbeat, and honeypot heartbeat status.

Interview explanation. *FastAPI fits a typed Python service: Pydantic validates boundary data, dependency injection centralizes authentication, and OpenAPI is available during development. It is not selected because it magically secures the system; the routes and dependencies implement the controls.*

Chapter 7

Authentication

What is it? Authentication answers “who is this user?” Authorization answers “may this user perform this operation?” The project uses email/password login, Argon2-compatible `pwdlib` password hashes, JWT access tokens, stored hashed refresh tokens, and one-time reset tokens.

How it works here. Login rate-limits an IP/email identity, verifies the hash, writes an audit log, then issues a 15-minute HS256 access JWT containing subject, role, issued-at, expiry, and type. A refresh token is a random value; only its SHA-256 digest is stored. Refresh rotates by revoking the presented record before issuing a new pair. Logout revokes the matching refresh token. The Next BFF uses HTTP-only cookies; `COOKIE_SECURE=true` is required for an HTTPS production deployment.

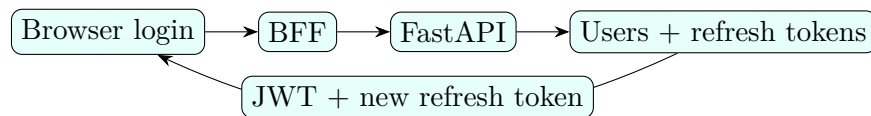


Figure 7.1: Login and token issuance

Chapter 8

Role-Based Access Control

What is it? The actual roles are ADMIN, ANALYST, and VIEWER. All management routes require a valid access token.

How it works here. `current_user` decodes the JWT and loads an active user. `require_roles` enforces role checks server-side. ADMIN/ANALYST can update incidents and export data; ADMIN creates/updates users and alert rules. A VIEWER may read permitted management data but receives 403 on administration operations. The frontend navigation is a convenience, not the security boundary.

Interview explanation. *Hiding a button is not authorization: a user can call the API manually. The backend must decide.*

Chapter 9

Password Reset

How it works here. A request returns the same generic 202 response whether or not an account exists, reducing account enumeration. For an active user, it generates a random token, stores only its SHA-256 digest with a 30-minute expiry, marks every prior unused reset token as used, and optionally sends the raw token to a private webhook. Confirmation is rate-limited, rejects missing/used/expired tokens, changes the password hash, marks the reset token used, and revokes all active refresh tokens.

9.1 Webhook signing

When configured, the webhook payload is canonical JSON with `event`, `delivery_id`, email, reset token, and issue time. The sender computes $\text{HMAC-SHA256}(\text{timestamp} . \text{body}, \text{secret})$, sends timestamp/signature/delivery-ID headers, and production configuration requires HTTPS and a secret. A receiving webhook must verify signature, freshness, and delivery-ID uniqueness; a receiver is **not implemented in this repository**, and real delivery was not tested.

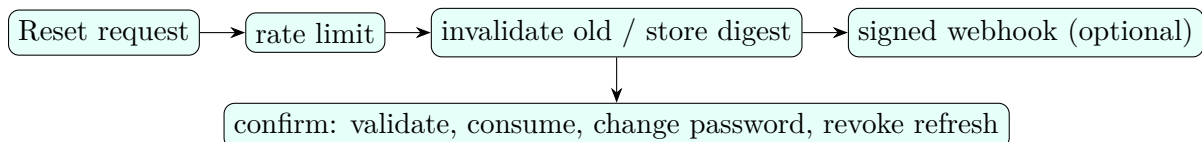


Figure 9.1: Password-reset flow

Chapter 10

Rate Limiting

What is it? Rate limiting caps repeated attempts within a time window, reducing password guessing and reset abuse.

How it works here. Login (default 10), reset request (5), and reset confirmation (10) use Redis fixed-window keys over the configurable 900-second window. Keys hash `source_ip|subject`, so emails are not placed in Redis key names. INCR counts and first use sets expiry. When Redis is unavailable, a process-local deque provides basic protection; it is weaker in multi-instance deployments and health endpoints reveal the outage. Ingestion has a separate per-source per-minute limiter, default 600.

Chapter 11

Redis and Redis Streams

What is it? Redis is an in-memory data store used here for short-lived coordination, not the source of security evidence.

Why do we need it? PostgreSQL is appropriate for relational, durable incident evidence, whereas Redis is quick for counters, heartbeats, pub/sub, and queued jobs.

How it works here. After committing an event, FastAPI uses `XADD honeypot:jobs` with `kind=enrich_event`, event ID, and source IP; it also publishes a lightweight notification. The worker creates consumer group `workers`, reads as a named consumer, acknowledges successful messages with `XACK`, and sets a 15-second heartbeat. Native validation observed stream entries, one consumer, and zero pending/lag.

Chapter 12

Worker

What is it? The worker is a separate Python process for work that should not delay the sensor ingestion response.

How it works here. For each job, it (1) caches/enriches an IP using an optional configured provider, (2) loads the persisted ML model and writes a prediction for the incident if applicable, and (3) creates in-app notification state and optionally posts an alert webhook. Failures are logged and unacknowledged messages remain available for investigation.

Interview explanation. *Direct API processing is simpler but ties decoy responsiveness to external provider latency and model work. Redis Streams isolate that workload and permit consumer scaling.*

Chapter 13

PostgreSQL, Migrations, and Data Model

What is it? PostgreSQL is the authoritative transactional record. Alembic migrations 0001 and 0002 build the schema; native validation reached revision 0002. SQLAlchemy models define fields, foreign keys, relationships, unique constraints, and indexes.

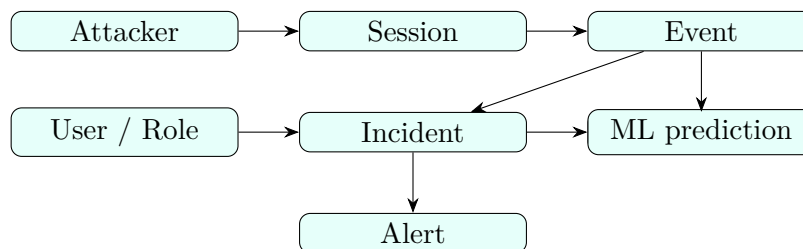


Figure 13.1: Simplified implemented entity relationships

Table 13.1: Important persisted entities

Entity	Purpose and key relationships
roles/users	Unique role name; user email is unique and indexed; a user belongs to a role.
refresh_tokens/password_digests/tokens	Token refresh tokens, expiry, and revocation/used timestamps; reset and refresh reference a user.
honeypot_nodes/attacks/sessions	Sensor identity, IP identity/enrichment, and a protocol session joining attacker to node.
events	UUID event, timestamp, IP, protocol, payload, severity and score. Indexed by timestamp/type and IP/timestamp.
command_events/http_requests/credential_capture	One request/credential capture event. Credential records preserve a fingerprint; ciphertext is nullable.
detections	Rule ID, attack type, score, and evidence for an event.
incidents/incident_events	Assigned IP-based case and its many-to-many event links.
alerts/alert_rules/notifications	Alert lifecycle, configurable threshold rules, and delivery state.
threat_intel/ml_predictions	Caches and logs, persisted prediction inputs/results, and security-relevant user actions.

Chapter 14

SSH Honeypot

What is it? The AsyncSSH service is a decoy that accepts interaction and exposes a deterministic virtual shell.

How it works here. It records authentication/commands and sends normalized events through the ingestion API. The shell dispatches an allowlist of simulated commands and response text; it does not call a host shell, subprocess, `eval`, or `os.system`. This boundary is essential: an attacker string such as a host escape must be telemetry, never execution. Native validation included a negative host-escape check.

Implementation status: The decoy is implemented and locally validated. It was not deployed to a public Internet host; public exposure requires separate isolation and host review.

Chapter 15

HTTP Honeypot and Ingestion Pipeline

What is it? The HTTP decoy records method, target, query, redacted headers, body size, response code, and session identity. It is useful because scanners and exploit attempts often arrive over HTTP.

How it works here. Both sensors submit the common event envelope to `POST /api/v1/ingest/events` with `X-Sensor-Key`. Pydantic validates fields; body limits and sensor-key comparison protect the boundary. The API creates protocol-specific rows and detects paths, query strings, body previews, commands, and user agents. Sensor heartbeats register/update node name, kind, port, and last-seen time.

Listing 15.1: Actual sensor authentication dependency, condensed

```
expected = get_settings().sensor_api_key.get_secret_value()
if not x_sensor_key or not hmac.compare_digest(x_sensor_key, expected):
    raise HTTPException(status_code=401, detail="Invalid_sensor_credentials")
```

Chapter 16

Detection Engine

What is it? Rules are deterministic, synchronous, and explainable. They inspect the normalized event plus a 60-second context.

Table 16.1: All 13 implemented detection rules

Rule	Condition	Score	Action/type
AUTH-BRUTE-001	At least five auth failures in 60 seconds	55	BRUTE_FORCE
SSH-CMD-WGET	<code>wget</code> command token	35	suspicious command
SSH-CMD-CURL	<code>curl</code> command token	30	suspicious command
SSH-CMD-CHMOD	<code>executable/777 chmod</code>	35	suspicious command
SSH-CMD-REVERSE_SHELL	interactive bash, netcat listener, or <code>/dev/tcp</code>	60	suspicious command
SSH-CMD-SCRIPT_EXEC	Python/Perl/Ruby inline execution	45	suspicious command
CRED-ACCESS-001	Credential/secret path term	40	credential access
WEB-SCAN-001	Sensitive web path; higher after 3 probes	20/35	web scanning
WEB-SQLI-001	SQL injection expression	55	SQL injection
WEB-TRAV-001	Decoded <code>../</code> or <code>..\</code>	50	path traversal
WEB-CMDI-001	Shell metacharacter expression	55	command injection
WEB-UA-001	Known scanner user agent	25	scanner
WEB-RATE-001	At least 30 HTTP requests in 60 seconds	45	high-frequency scanning

Implementation status: All 13 rules were exercised through live authenticated ingestion in native validation. Rules are pattern/context detections, not a complete exploit classifier.

Chapter 17

Scoring and Incident Correlation

How it works here. For matched scores sorted high to low $s_1, s_2, \dots$, event risk is $\min(100, s_1 + \sum_{i>1} \text{round}(0.35s_i))$. Severity is LOW 0–29, MEDIUM 30–59, HIGH 60–79, CRITICAL 80–100. This is visible, auditable logic rather than a hidden model.

How it works here. A detection event joins the newest unresolved incident with the same source IP and last-seen time inside 15 minutes. A new incident starts at the event score. A later event takes the greater old/new score plus $\min(10, 2 \times \text{matched rules})$, capped at 100; attack types become a sorted union. Resolved incidents do not receive correlation.

Interview explanation. *An event is raw evidence; an incident is the analyst's correlated case. For example, traversal and SQL injection from one IP can become one unresolved incident rather than two disconnected tickets.*

Chapter 18

Alerts and Notifications

How it works here. Without configured alert rules, an incident alerts at score at least 80 or for brute force. Configured enabled rules can set a minimum score and optional attack type. The system updates an existing unresolved alert instead of duplicating it. The worker marks an IN_APP notification delivered and can attempt an optional webhook. Incident acknowledgement/resolution propagates status/timestamps to linked alerts and writes an audit record.

Chapter 19

Server-Sent Events

What is it? Real-time communication lets an analyst see new telemetry without a manual reload. Polling repeatedly asks for changes; WebSockets are bidirectional persistent channels; SSE is a simple server-to-browser HTTP stream.

Why do we need it? This dashboard primarily needs one-way telemetry notifications. SSE uses browser `EventSource`, is simpler than a two-way protocol, and works with the existing API stream.

How it works here. `GET /api/v1/management/stream/events` reads events newer than a cursor once per second, emits `event: telemetry` JSON frames, and sends keepalive comments. The client invalidates cached queries and reconnects after errors. Native Playwright validation verified a live count update and SSE reconnect/recovery. The deliberate API-failure dashboard UI browser scenario remains only partially tested.

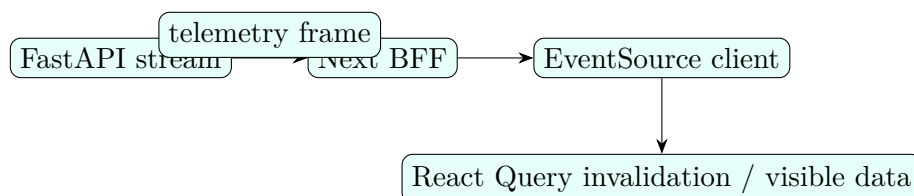


Figure 19.1: SSE update path

Chapter 20

Machine-Learning Pipeline

What is it? The ML component is an executable logistic-regression baseline, not the primary detection engine.

How it works here. The training module generates explicitly synthetic development data, fits a persisted joblib model and writes evaluation JSON. The worker extracts aggregate incident features from linked events, loads the model, calls `predict_proba`, and stores classification, confidence, model version, and exact features in `ml_predictions`. Logistic regression estimates $p(y = 1|x) = 1/(1 + e^{-(w^T x + b)})$; it is interpretable and lightweight.

Implementation status: Model training, persistence, worker load/prediction, and database persistence were validated. The supplied data is intentionally separable synthetic development data. Its metrics do **not** measure real-world attack detection accuracy.

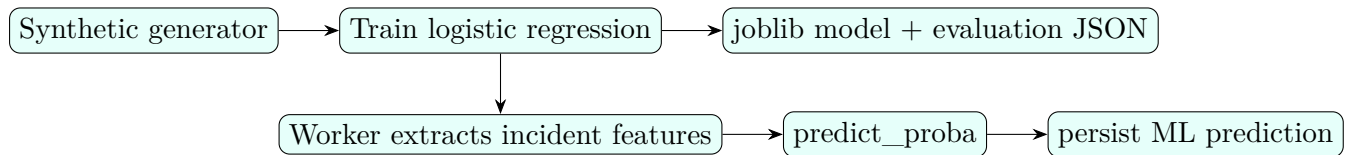


Figure 20.1: Implemented ML lifecycle

Chapter 21

Threat Intelligence

What is it? Threat intelligence enriches an observable, here an IP, with external reputation/location data.

How it works here. The worker uses one generic configured HTTP provider endpoint and bearer key, retries up to three times with short backoff, caches the result for 24 hours, and isolates errors. When available it populates attacker country/city/coordinates/ASN/organization; when unavailable it persists status rather than breaking the job.

Implementation status: The no-provider path was exercised and persisted UNAVAILABLE records. No provider credentials were supplied, so external provider responses were not tested. There are no provider-specific adapters.

Chapter 22

Security Architecture

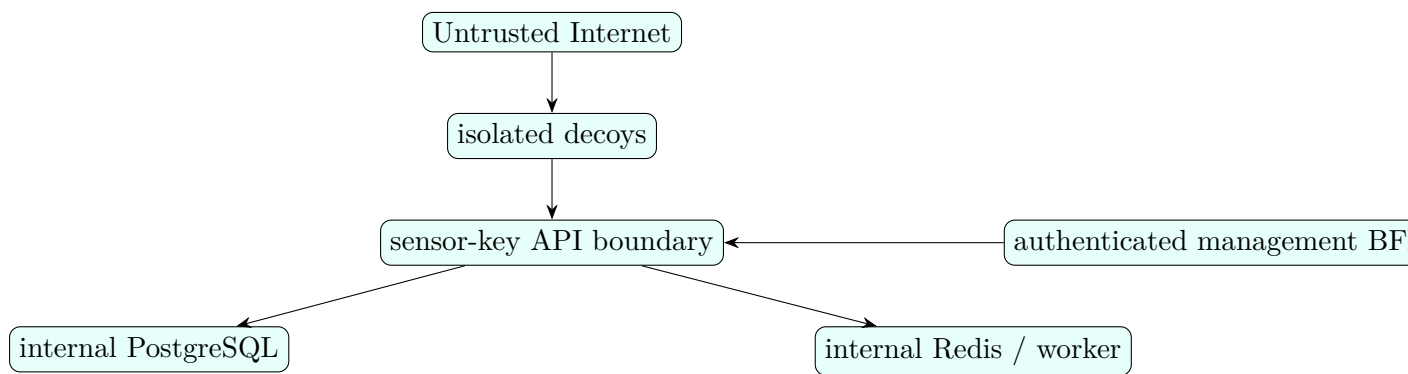


Figure 22.1: Security boundaries: only intended interfaces cross layers

Table 22.1: Threat-to-defense mapping

Threat	Implemented defense
Credential guessing/reset abuse	Redis-backed rate limits; generic reset response; Argon2-compatible password verification.
Access-token misuse	Short-lived typed JWT; backend lookup of active user; refresh rotation/revocation.
Unauthorized role action	Backend <code>require_roles</code> , not just hidden UI controls.
Webhook forgery	HMAC-SHA256 over timestamp and canonical body; production HTTPS/secret validation. Receiver replay enforcement is external/not implemented.
Sensor impersonation	Constant-time sensor-key comparison. Shared key is a documented limitation.
Host compromise via decoy input	SSH allowlisted virtual shell; no host command execution; container hardening configuration exists.
Oversized/frame/referrer risks	Content limits; nosniff, DENY frame, and no-referrer headers.
Data-service exposure	Intended internal network segmentation and loopback native bindings. Public deployment is not performed.

22.1 Containers and secrets

Docker configuration uses non-root users, dropped capabilities, no-new-privileges, resource limits, segmented networks, and read-only filesystems where practical. Docker/TLS runtime was **not tested**. Secrets come from environment variables and must never be committed; production requires unique database password, database/Redis URLs, JWT secret, sensor key, CORS origin, secure cookies, and any webhook secret.

Chapter 23

Error Handling and Resilience

How it works here. The frontend converts fetch failures/timeouts to a recoverable 503-style error and supplies retry UI. SSE provides visible status and reconnection. Ingestion commits PostgreSQL before best-effort Redis enqueue, preserving authoritative event state during a Redis outage; rate limiting has a local fallback. Health endpoints expose database, Redis, worker heartbeat, and honeypot heartbeat. External threat intelligence errors become cached ERROR/UNAVAILABLE state instead of failing ingestion. The worker logs job errors and only acknowledges successful messages.

Implementation status: Code and focused validation cover these paths. A deliberate browser API-outage UI test remains partially tested; the dashboard should be tested against an actual production failure mode after deployment.

Chapter 24

Technology Stack and Rationale

Table 24.1: Actual technology stack

Technology	Purpose	Why it fits here	Alternative
Next.js 16 / React 19	SOC UI/BFF	Route structure, React UI, server/client boundaries	Plain React SPA
TypeScript	UI correctness	Typed API/UI contracts and refactoring help	JavaScript
Tailwind / Recharts	Styling/charts	Fast consistent styling and dashboard visualizations	CSS modules/chart library
FastAPI / Pydantic	API/validation	Typed Python routes, validation, dependencies, OpenAPI	Flask
SQLAlchemy / Alembic	ORM/schema evolution	Relational mapping and reproducible migrations	Raw SQL/manual schema
PostgreSQL	Durable evidence	Transactions, foreign keys, indexes, relational investigations	MongoDB
Redis / Streams	Jobs, counters, heartbeats	Low-latency streams and ephemeral coordination	Direct synchronous work
AsyncSSH	SSH decoy	Async protocol server with controlled shell integration	Custom SSH protocol
scikit-learn / joblib	ML baseline	Mature simple logistic regression/persistence	Heavier ML platform
Pytest / Playwright	Testing	Python unit/integration and browser workflow coverage	Other test runners
Ruff / ESLint / TypeScript compiler	Static quality	Fast Python and frontend checks	Manual review only
Docker Compose / GitHub Actions	Deployment/CI configuration	Repeatable service topology and automation	Manual process management

Interview explanation. *PostgreSQL is chosen for durable relationships and incident workflow; Redis complements it for fast ephemeral coordination. Next.js adds a BFF and application routing beyond a plain client-only React app. SSE suits one-way updates; WebSockets would be justified only for richer bidirectional realtime interaction.*

Chapter 25

Project Directory Structure

Table 25.1: Important repository areas

Path	Role and dependencies
<code>apps/api/app</code>	FastAPI routers, models, schemas, security, detector rules, services; depends on PostgreSQL and Redis.
<code>apps/api/migrations</code>	Alembic revision history; deploy with <code>alembic upgrade head</code> .
<code>apps/worker</code>	Redis Stream consumer; uses API models and <code>ml</code> .
<code>apps/ssh-honeypot</code>	AsyncSSH listener and deterministic virtual shell.
<code>apps/http-honeypot</code>	FastAPI HTTP decoy which normalizes requests.
<code>apps/web/app</code>	Next routes, including BFF route handlers and protected pages.
<code>apps/web/components,lib</code>	Dashboard UI, SSE listener, API client, types/utilities.
<code>ml</code>	Synthetic generator, feature extraction, training, evaluation, prediction/model artifact.
<code>packages/sensor-sdk</code>	Shared sensor-side event support (used by decoys).
<code>scripts</code>	Native Windows setup/start, seed, data generation, and validation scripts.
<code>docs</code>	Detection, security, verification, interview, and audit records.
<code>docker-compose*.yaml</code>	Optional container deployment/hardening topology; not used in native validation.

Chapter 26

API Documentation

Table 26.1: Important implemented API surface

Route	Access	Purpose/security notes
POST /api/v1/auth/register	Public	Creates user; bootstrap configured first matching email becomes ADMIN.
POST /api/v1/auth/login	Public + rate limit	Validates credentials, logs login, returns access/refresh pair.
POST /api/v1/auth/refresh	Refresh token	Revokes presented stored token and rotates pair.
POST /api/v1/auth/logout	Refresh token	Revokes known refresh record; 204.
POST /api/v1/auth/password-reset/request	Public + rate limit	Generic 202; one-time token only for valid active user.
POST /api/v1/auth/password-reset/confirm	Public + rate limit	Consumes valid token, changes password, revokes sessions.
POST /api/v1/ingest/heartbeat	Sensor key	Upserts decoy node health.
POST /api/v1/ingest/events	Sensor key + limits	Validates/idempotently persists event, detections, incident/alert.
GET /api/v1/management/events	JWT	Filtered, paginated evidence listing.
GET/PATCH /management/incidents	JWT / ADMIN or ANALYST patch	Investigation and lifecycle changes with audit log.
GET /management/stream/events	JWT	SSE telemetry stream.
GET /management/analytics/overview	JWT	Aggregates for dashboard charts/cards.
/management/admin/*	Role dependent	Users/rules/audit logs; mutations require ADMIN.
/health/*	Deployment-controlled	API, database, Redis, worker, and hon-eypot status.

Implementation status: Development OpenAPI is available at /docs; it is disabled when ENVIRONMENT=production. Exact schemas live in apps/api/app/schemas.

Chapter 27

Testing and Evidence

Table 27.1: Recorded validation evidence

Check	Recorded result
Ruff	PASS.
Pytest	PASS: 18 tests after auth/reset hardening.
Alembic native PostgreSQL	PASS at revision 0002.
ESLint / TypeScript / Next production build	PASS.
Playwright main workflows	PASS: login, ingestion, investigation, acknowledgement, resolution, major SOC routes.
SSE	PASS: live update and reconnect/recovery browser coverage.
Native services	PostgreSQL, Redis, API, worker, SSH and HTTP decoys validated.
Detection	All 13 deterministic rules validated through live ingestion.
ML	Training, persistence, worker prediction, persistence validated.
Docker/TLS runtime	Not run; Docker intentionally not installed locally.
External intelligence/reset delivery	Not tested without provider/webhook credentials.
Controlled API-failure browser UI	Partially tested; no final deterministic browser outage assertion.

How it works here. Unit tests cover API/auth/detection/decoys/features. Native scripts validate honeypots, detection pipeline, and auth runtime. Playwright covers a browser workflow against the native stack. The report must not be read as an independent penetration test or production deployment validation.

Chapter 28

Deployment

Implementation status: Current status: **not deployed**. There is no public HTTPS URL and no cloud-provider account/credential configured in the validation environment.

How it works here. A future deployment should provision private PostgreSQL and Redis, API, worker, and dashboard services. Run `alembic upgrade head` against the managed database. Configure unique secrets from `.env.example`: database/Redis URLs, JWT secret, sensor API key, production CORS origin, secure cookies, optional provider/webhook secrets, and model path. Place an HTTPS reverse proxy or provider TLS endpoint before the management plane. Do not expose PostgreSQL, Redis, worker, secrets, or internal management services publicly. Deploy decoys only on separately reviewed, isolated infrastructure; their host must never execute attacker input.

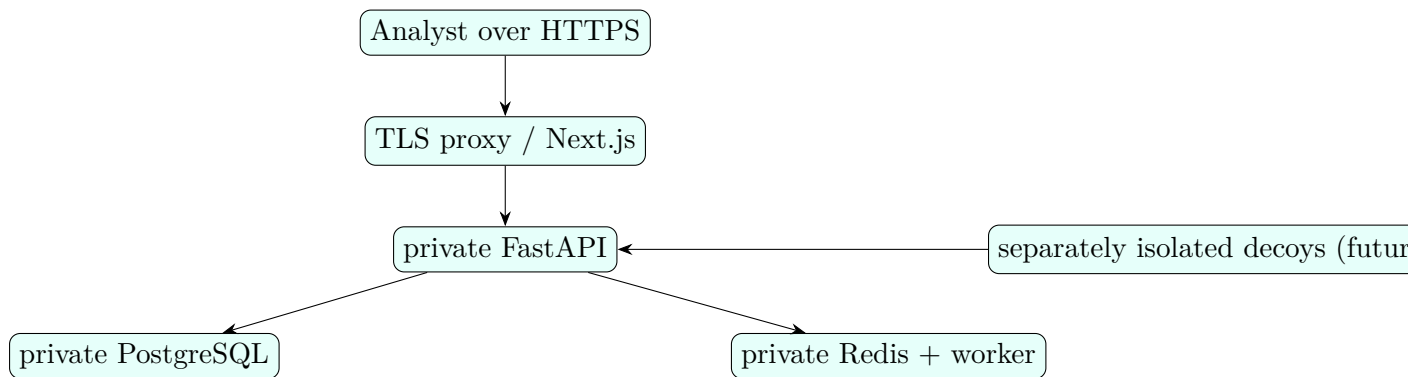


Figure 28.1: Target deployment architecture, not a statement of current deployment

Chapter 29

Limitations

- The ML dataset is synthetic; accuracy is not real-world evidence.
- External threat intelligence was not provider-tested.
- Password-reset webhook delivery and receiver-side replay enforcement are not implemented/tested end-to-end.
- The dashboard API-failure UI is only partially browser-tested.
- Docker/TLS container runtime was not tested.
- The platform is not publicly deployed and is not independently penetration tested.
- Sensor authentication is a shared key; per-sensor identities/rotation are future hardening.
- FTP, Telnet, and MySQL decoys are extension points, not implemented services.

Chapter 30

Future Improvements

Implementation status: Everything in this chapter is planned/future work, not implemented functionality.

- Deploy with private managed data services, HTTPS, backups, monitoring, and a security review.
- Add per-sensor credentials, rotation, mTLS or signed envelopes.
- Build and test a reset-webhook receiver with timestamp age and delivery-ID replay storage.
- Integrate and evaluate real threat-intelligence providers.
- Train/evaluate with governed real-world telemetry and perform drift monitoring.
- Improve correlation across indicators, add queues/metrics/tracing, and integrate with a SIEM.
- Expand safely isolated sensor coverage and distributed worker recovery policy.

Chapter 31

Interview Preparation

30-second explanation

HoneyPot System is a SOC-style honeypot platform. Safe SSH and HTTP decoys collect attacker actions, FastAPI validates and scores them with deterministic rules, PostgreSQL stores evidence and incidents, Redis sends enrichment jobs to a worker, and a Next.js dashboard gives analysts live SSE updates.

1-minute explanation

The key design decision is separation: decoys have no database credentials and the SSH shell never executes host commands. Ingestion is authenticated and idempotent. The API synchronously evaluates explainable rules and correlates a source IP's related detections within 15 minutes; the worker handles slower enrichment, ML prediction, and notification. Users get short-lived JWT access plus rotating stored refresh tokens, role checks are enforced at the API, and login/reset operations are rate-limited.

2-minute explanation

Explain the full flow from decoy interaction to event persistence, rules, score, correlation, Redis job, worker, and SSE dashboard refresh; then call out that rules are primary and ML is a synthetic-data baseline. State clearly that it is locally validated but not publicly deployed.

5-minute technical explanation

Discuss transactional PostgreSQL persistence before best-effort Redis enqueue, consumer groups/acknowledgement, HMAC reset delivery, backend RBAC, error/retry behavior, and the 13 detection rules. Finish with limitations and deployment isolation requirements.

Table 31.1: Interview questions and accurate answers

Question	Answer
What exactly does the honeypot do?	It simulates SSH/HTTP services, captures interactions as telemetry, and never executes attacker commands on the host.
Why Redis?	Streams decouple background work; counters/expiry support rate limits; heartbeat supports health. PostgreSQL remains authoritative.

Question	Answer
Why PostgreSQL?	Evidence has strong relationships, transactions, indexes, lifecycle updates, and audit requirements.
Why FastAPI?	Typed validation, dependency-based security, async-friendly web framework, and development OpenAPI.
Why Next.js?	It structures the dashboard and supplies a BFF so browser management calls need not directly expose backend token handling.
How does an event become an incident?	Ingestion validates/persists it, rules produce detections/score, then matching unresolved same-IP detections within 15 minutes correlate into an incident.
Why SSE rather than WebSockets?	The implemented requirement is server-to-browser telemetry; SSE/EventSource is simpler for that one-way stream.
How is password reset secured?	Generic response, rate limits, random token digest storage, expiry, one-time consumption, old-token invalidation, refresh revocation, optional HMAC-signed delivery.
What happens if Redis fails?	Event persistence still succeeds; background enqueue may be skipped, local rate-limit fallback applies, and health reports the outage.
What is the ML model/data?	Persisted logistic regression trained on synthetic development data; it validates plumbing, not real-world performance.
Is it deployed?	No. The repository was locally validated; no public HTTPS deployment was performed.

Chapter 32

Glossary

Table 32.1: Glossary

Term	Definition
SOC	Security Operations Center: the function that monitors and responds to security telemetry.
SIEM	Security Information and Event Management; centralized collection/search/correlation tooling.
Honeypot	Controlled decoy intended to observe unwanted interaction.
Telemetry	Structured observations emitted by a system or sensor.
IOC	Indicator of compromise, an observable clue such as an IP or malicious path.
RBAC	Role-Based Access Control: authorization based on assigned role.
JWT	Signed JSON Web Token used here for short-lived access.
SSE	Server-Sent Events: one-way HTTP streaming from server to browser.
Redis Stream	Append-only Redis message stream with consumer-group support.
Worker	Separate process that consumes queued work.
ORM	Object-Relational Mapping; SQLAlchemy maps Python models to tables.
Alembic	Migration tool that evolves the SQL schema reproducibly.
CORS	Browser cross-origin policy configured by the API.
HMAC	Keyed message authentication code used to authenticate webhook content.
Replay attack	Reuse of a previously valid message/token; mitigated by expiry, one-time state, timestamps, and delivery IDs.
Rate limiting	Restricting requests per identity/window to slow abuse.
Logistic regression	A probabilistic linear classifier used by the synthetic ML baseline.
Inference	Applying a trained model to extracted incident features.

Source Basis and Reading Order

This handbook was written from the repository's current source and documentation, especially `apps/api`, `apps/worker`, decoy services, `apps/web`, Alembic migrations, ML modules, `README.md`, `docs/DETECTION.md`, `docs/SECURITY.md`, `docs/VERIFICATION.md`, scripts, and tests. For a first reading, use Chapters 1–4, 7–13, 17–21, 24, 31, and 35. For code-level preparation, open the referenced source paths alongside the appropriate chapter.